МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Львівський національний університет імені Івана Франка

ЗВІТ

до лабораторної роботи №7
з дисципліни «Професійний Python»

Тема: «Робота з базами даних у Python»

Виконав:
студент Левандівський Назар

Перевірив:
Куньо Іван Михайлович

Львів – 2026

Мета роботи

Ознайомитися з організацією шару зберігання даних у Python-застосунках: реляційна СУБД SQLite, мова SQL, стандарт DB-API, параметризовані запити, CRUD, транзакції та властивості ACID, Repository Pattern, SQLAlchemy ORM і міграції Alembic. Інтегрувати persistence layer у проєкт студентів і перевірити його integration tests на окремій тестовій базі.

1. Постановка задачі та структура проєкту

Варіант 1. Система обліку студентів. Предметну область ЛР1 збережено: студент має ім'я, прізвище, групу та середній бал (0–100). Потрібно: пов'язані таблиці `groups` і `students` (one-to-many), `StudentRepository` і `GroupRepository`; функції ЛР1 (додавання й виведення, пошук за прізвищем і групою, найкращий студент, середній бал групи) плюс зміна бала, переведення між групами та видалення; параметризований пошук за групою через DB-API; транзакція масового переведення з commit/rollback; міграція, що додає `email`; integration tests. Архітектура: CLI → `StudentService` → репозиторії → SQLAlchemy ORM → SQLite; `dbapi.py` працює з SQLite напряму через `sqlite3`.

```
Lab_07/
├── pyproject.toml        залежності: sqlalchemy, alembic; [test]/[dev]: pytest, pytest-cov, mypy, ruff
├── alembic.ini           конфігурація Alembic
├── migrations/           env.py; versions/0001_create_groups_and_students.py, 0002_add_student_email.py
├── src/student_manager/
│   ├── database.py       engine, PRAGMA foreign_keys = ON, фабрика сесій
│   ├── models.py         ORM-моделі Group і Student, перевірка полів (validates)
│   ├── repositories.py   Generic Repository; GroupRepository, StudentRepository
│   ├── services.py       StudentService, транзакція transfer_students
│   ├── dbapi.py          пошук студентів групи через sqlite3 (DB-API)
│   ├── validation.py     правила для імені, групи, балу, email; exceptions.py
│   └── main.py           CLI student-manager
├── tests/                conftest.py + 10 модулів integration tests (212 тестів)
├── experiments/          run_experiments.py -> results/experiments.json
└── tools/sql.py          SQL-консоль для SQLite (утиліти sqlite3 у Windows немає)
```

2. Схема бази даних

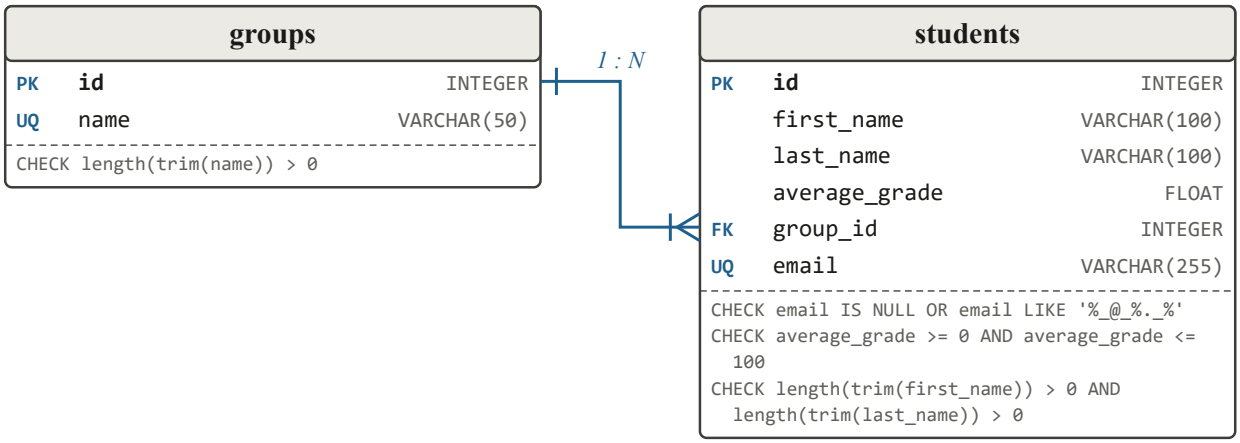


Рисунок 1 – ER-діаграма (побудована з метаданих ORM-моделей); зовнішній ключ із ON DELETE CASCADE

Таблиця 1 – Таблиці, ключі та поля

Таблиця	Ключі	Поля
groups – групи	PK id; UNIQUE name	name – код групи, наприклад PI-21 (CHECK: не порожнє)
students – студенти	PK id; FK group_id → groups.id; UNIQUE email; індекси group_id, last_name	first_name, last_name – ім'я та прізвище (CHECK: не порожні); average_grade – середній бал (CHECK: 0…100); group_id – група; email – адреса, може бути NULL (додано міграцією 0002, CHECK: формат)

Таблиці створює міграція Alembic. DDL, яку SQLite зберігає в `sqlite_master` після `alembic upgrade head` (students перебудовано міграцією 0002, тому її назва в лапках):

```
CREATE TABLE groups (
  id INTEGER NOT NULL,
  name VARCHAR(50) NOT NULL,
  CONSTRAINT pk_groups PRIMARY KEY (id),
  CONSTRAINT ck_groups_name_not_blank CHECK (length(trim(name)) > 0),
  CONSTRAINT uq_groups_name UNIQUE (name)
);
CREATE TABLE "students" (
  id INTEGER NOT NULL,
  first_name VARCHAR(100) NOT NULL,
  last_name VARCHAR(100) NOT NULL,
  average_grade FLOAT NOT NULL,
  group_id INTEGER NOT NULL,
  email VARCHAR(255),
  CONSTRAINT pk_students PRIMARY KEY (id),
  CONSTRAINT fk_students_group_id_groups FOREIGN KEY(group_id) REFERENCES groups (id) ON DELETE CASCADE,
  CONSTRAINT ck_students_grade_in_range CHECK (average_grade >= 0 AND average_grade <= 100),
  CONSTRAINT ck_students_names_not_blank CHECK (length(trim(first_name)) > 0 AND length(trim(last_name)) > 0),
  CONSTRAINT uq_students_email UNIQUE (email),
  CONSTRAINT ck_students_email_format CHECK (email IS NULL OR email LIKE '%_@_%._%')
);
CREATE INDEX ix_students_group_id ON students (group_id);
CREATE INDEX ix_students_last_name ON students (last_name);
```

3. Підключення до БД (database.py) і ORM-моделі

SQLite за замовчуванням не перевіряє FOREIGN KEY, тому прагма вмикається для кожного нового з'єднання через подію `connect` (експеримент 2). URL береться зі змінної `STUDENT_MANAGER_DATABASE_URL` або `sqlite:///students.db`. Сесія створюється з `autoflush=False` (SQL надсилається явно) і `expire_on_commit=False` (об'єкти лишаються читабельними після commit).

```
def make_engine(url: str | None = None, *, echo: bool = False) -> Engine:
    engine = create_engine(url or database_url(), echo=echo)
    if engine.dialect.name == "sqlite":
        event.listen(engine, "connect", _enable_foreign_keys)
    return engine

def _enable_foreign_keys(dbapi_connection: Any, _connection_record: Any) -> None:
    cursor = dbapi_connection.cursor()
    cursor.execute("PRAGMA foreign_keys = ON")
    cursor.close()

def make_session_factory(engine: Engine) -> sessionmaker[Session]:
    return sessionmaker(bind=engine, autoflush=False, expire_on_commit=False)
```

Моделі написано в стилі SQLAlchemy 2 (`Mapped`, `mapped_column`). `relationship()` з `back_populates` задає one-to-many; `cascade` і `ondelete="CASCADE"` – каскадне видалення. Значення перевіряються двічі: у `@validates` (до звернення до БД) і CHECK-обмеженнями (навіть якщо запис іде повз ORM).

```
class Group(Base):
    __tablename__ = "groups"
    __table_args__ = (CheckConstraint("length(trim(name)) > 0", name="name_not_blank"),)

    id: Mapped[int] = mapped_column(primary_key=True)
```

```
    name: Mapped[str] = mapped_column(String(50), unique=True)

    students: Mapped[list["Student"]] = relationship(
        back_populates="group",
        cascade="all, delete-orphan",
        passive_deletes=True,
        order_by="Student.id",
    )


class Student(Base):
    __tablename__ = "students"
    __table_args__ = (
        CheckConstraint("average_grade >= 0 AND average_grade <= 100", name="grade_in_range"),
        CheckConstraint(
            "length(trim(first_name)) > 0 AND length(trim(last_name)) > 0",
            name="names_not_blank",
        ),
        CheckConstraint("email IS NULL OR email LIKE '%_@_%._%'", name="email_format"),
    )

    id: Mapped[int] = mapped_column(primary_key=True)
    first_name: Mapped[str] = mapped_column(String(100))
    last_name: Mapped[str] = mapped_column(String(100), index=True)
    average_grade: Mapped[float] = mapped_column(Float)
    group_id: Mapped[int] = mapped_column(ForeignKey("groups.id", ondelete="CASCADE"), index=True)
    email: Mapped[str | None] = mapped_column(String(255), unique=True, nullable=True)

    group: Mapped[Group] = relationship(back_populates="students")

    @validates("average_grade")
    def _validate_grade(self, _key: str, value: object) -> float:
        return clean_grade(value)
```

4. Repository Pattern

Generic Repository містить CRUD, спільний для всіх моделей; конкретні репозиторії додають запити предметної області: пошук за прізвищем, фільтрацію за групою (з JOIN), пошук з фільтрами, сортуванням і пагінацією, `AVG`, `COUNT`, `MIN`/`MAX` з `GROUP BY`. Репозиторії викликають лише `flush()`, а не `commit()`: межі транзакції визначає сервіс. Фрагмент:

```
class Repository(Generic[ModelT]):
    model: type[ModelT]

    def add(self, entity: ModelT) -> ModelT:
        self.session.add(entity)
        self.session.flush()
        return entity

    def get_by_id(self, entity_id: int) -> ModelT | None:
        return self.session.get(self.model, entity_id)


class StudentRepository(Repository[Student]):
    model = Student

    def find_by_last_name(self, last_name: str) -> list[Student]:
        statement = (
            select(Student)
            .where(Student.last_name == last_name.strip())
            .order_by(Student.first_name, Student.id)
        )
        return list(self.session.scalars(statement))

    def update_grade(self, student_id: int, grade: float) -> Student | None:
        student = self.get_by_id(student_id)
        if student is None:
            return None
        student.average_grade = grade
        self.session.flush()
        return student


class GroupRepository(Repository[Group]):
    model = Group

    def stats(self) -> list[GroupStats]:
        statement = (
            select(
                Group.name,
                func.count(Student.id),
                func.avg(Student.average_grade),
```

```
                func.max(Student.average_grade),
                func.min(Student.average_grade),
            )
            .outerjoin(Student, Student.group_id == Group.id)
            .group_by(Group.id, Group.name)
            .order_by(Group.name)
        )
        return [
            GroupStats(
                name,
                int(students),
                None if average is None else float(average),
                None if best is None else float(best),
                None if worst is None else float(worst),
            )
            for name, students, average, best, worst in self.session.execute(statement)
        ]
```

5. DB-API і параметризований запит

Пошук студентів групи виконано без ORM – через `sqlite3` (Connection → Cursor → `execute` → `fetchall`). Значення передається окремо від тексту SQL (плейсхолдер `?`), тому не може змінити структуру запиту. Рядок `PI-21' OR '1'='1`, вставлений через f-string, перетворює умову на `... OR '1'='1'` і запит повертає 9 студентів із 9; параметризований запит – 0 (експеримент 1).

```
FIND_BY_GROUP = """
    SELECT s.id, s.last_name, s.first_name, g.name, s.average_grade, s.email
    FROM students AS s
    JOIN groups AS g ON g.id = s.group_id
    WHERE g.name = ?
    ORDER BY s.last_name, s.first_name, s.id
"""


def find_students_by_group(database: Path | str, group_name: str) -> list[StudentRow]:
    with closing(sqlite3.connect(database)) as connection:
        cursor = connection.cursor()
        cursor.execute(FIND_BY_GROUP, (group_name,))
        rows = cursor.fetchall()
        cursor.close()
    return [StudentRow(*row) for row in rows]
```

6. CRUD і робота програми

Таблиця 2 – CRUD-операції

Операція	Методи StudentService / репозиторіїв	SQL
Create	`create_group`, `add_student`	INSERT
Read	`get_student`, `find_by_last_name`, `filter_by_group`, `list_students`, `get_group_average`, `get_best_student`, `group_report`	SELECT, JOIN, AVG, COUNT, MIN, MAX, GROUP BY
Update	`update_grade`, `set_email`, `transfer_students`	UPDATE
Delete	`delete_student`, `delete_group` (непорожню групу – лише з `force`, студенти видаляються каскадно)	DELETE

У ORM Update – це зміна атрибутів завантаженого об'єкта: сесія відстежує зміни (Unit of Work) і формує UPDATE під час flush. Сеанс роботи з CLI на новій базі (помилкові команди повертають код 1 і не змінюють даних):

```
PS C:\Github\prof_python\Lab_07> alembic upgrade head
INFO  [alembic.runtime.migration] Context impl SQLiteImpl.
INFO  [alembic.runtime.migration] Will assume transactional DDL.
INFO  [alembic.runtime.migration] Running upgrade  -> 0001
INFO  [alembic.runtime.migration] Running upgrade 0001 -> 0002
PS C:\Github\prof_python\Lab_07> student-manager seed
Створено груп: 3, студентів: 9
PS C:\Github\prof_python\Lab_07> student-manager groups
Група  Студентів  Середній   Макс    Мін
PI-21          3     86.17  95.00  78.00
PI-22          3     81.90  99.50  55.00
PI-23          3     73.83  88.00  62.50
PS C:\Github\prof_python\Lab_07> student-manager students --sort average_grade --desc --limit 4
 ID Студент             Група     Бал  Email
```

```
  4 Бондар Ірина         PI-22   99.50  iryna.bondar@example.com
  1 Коваленко Олена      PI-21   95.00  olena.kovalenko@example.com
  6 Гнатишин Марта       PI-22   91.20  -
  7 Гнатюк Софія         PI-23   88.00  -
PS C:\Github\prof_python\Lab_07> student-manager find Бондар
 ID Студент              Група     Бал  Email
  4 Бондар Ірина         PI-22   99.50  iryna.bondar@example.com
PS C:\Github\prof_python\Lab_07> student-manager dbapi-group PI-22
  4 Бондар Ірина 99.50
  6 Гнатишин Марта 91.20
  5 Гриценко Тарас 55.00
PS C:\Github\prof_python\Lab_07> student-manager add Назар Левандівський PI-21 88.5 --email nazar@example.com
Додано: 10 Левандівський Назар (PI-21, 88.5)
PS C:\Github\prof_python\Lab_07> student-manager grade 2 150
Помилка: середній бал 150.0 поза діапазоном 0–100
PS C:\Github\prof_python\Lab_07> student-manager transfer PI-23 1 2 999
Помилка: студента з id=999 не знайдено
PS C:\Github\prof_python\Lab_07> student-manager transfer PI-23 1 2
Переведено студентів: 2 -> PI-23
PS C:\Github\prof_python\Lab_07> student-manager groups
Група  Студентів  Середній   Макс    Мін
PI-21          2     87.00  88.50  85.50
PI-22          3     81.90  99.50  55.00
PI-23          5     78.90  95.00  62.50
PS C:\Github\prof_python\Lab_07> student-manager delete-group PI-23
Помилка: у групі PI-23 є студенти: 5
```

Рисунок 2 – Робота програми: міграція, наповнення, пошук, DB-API, додавання, невдалі оновлення й переведення, успішне переведення, спроба видалити непорожню групу

Вміст бази, переглянутий SQL-запитами:

```
PS C:\Github\prof_python\Lab_07> python tools/sql.py students.db "SELECT s.id, s.last_name, g.name, s.average_grade
FROM students s JOIN groups g ON g.id = s.group_id WHERE g.name = ? ORDER BY s.last_name" PI-23
| id | last_name | name | average_grade |
|---|---|---|---|
| 7 | Гнатюк | PI-23 | 88.0 |
| 8 | Дзюба | PI-23 | 62.5 |
| 1 | Коваленко | PI-23 | 95.0 |
| 2 | Мельник | PI-23 | 78.0 |
| 9 | Шевчук | PI-23 | 71.0 |
(5 рядків)
PS C:\Github\prof_python\Lab_07> python tools/sql.py students.db "SELECT g.name, COUNT(s.id) AS students,
ROUND(AVG(s.average_grade), 2) AS avg_grade FROM groups g LEFT JOIN students s ON s.group_id = g.id GROUP BY g.name"
| name | students | avg_grade |
|---|---|---|
| PI-21 | 2 | 87.0 |
| PI-22 | 3 | 81.9 |
| PI-23 | 5 | 78.9 |
(3 рядки)
```

Рисунок 3 – SQL-запити до students.db: JOIN із параметром і GROUP BY з COUNT та AVG

7. Транзакція transfer_students, commit і rollback, ACID

`transfer_students` переводить список студентів до іншої групи як єдине ціле. Після кожного студента виконується `flush()`, тож UPDATE справді надсилається в БД; якщо наступного студента не знайдено (або виникла помилка БД), `rollback()` скасовує вже надіслані зміни. Інші операції сервісу виконуються всередині `_transaction()` – контекстного менеджера з тією самою логікою.

```python
def transfer_students(session: Session, student_ids: Sequence[int], new_group_id: int) -> int:
    unique_ids = list(dict.fromkeys(student_ids))
    try:
        if not unique_ids:
            raise StudentValidationError("список студентів для переведення порожній")
        group = session.get(Group, new_group_id)
        if group is None:
            raise GroupNotFoundError(f"групу з id={new_group_id} не знайдено")
        for student_id in unique_ids:
            student = session.get(Student, student_id)
            if student is None:
                raise StudentNotFoundError(f"студента з id={student_id} не знайдено")
            student.group = group
            session.flush()
        session.commit()
```

```
    except Exception:
        session.rollback()
        raise
    return len(unique_ids)

@contextmanager
def _transaction(self) -> Iterator[None]:
    try:
        yield
        self.session.commit()
    except Exception:
        self.session.rollback()
        raise
```

Таблиця 3 – Як реалізовано властивості ACID

Властивість	**Реалізація в проєкті**	**Перевірка**
Atomicity	одна транзакція на операцію; rollback при будь-якому винятку	експ. 3: у списку [1, 2, 3, 777, 5] немає студента 777; commit після кожного кроку лишив 3 із 4 студентів у новій групі, DB-API з rollback і transfer_students – 0
Consistency	CHECK, UNIQUE, FOREIGN KEY + PRAGMA; правила в @validates і сервісі	тести test_constraints; експ. 2: без прагми лишилось 3 «осиротілих» студентів після видалення групи, з прагмою – каскадне видалення
Isolation	незафіксовані зміни не видно іншому з'єднанню	тест test_uncommitted_transfer_is_invisible_to_another_connection: після flush інше з'єднання ще бачить стару групу
Durability	після commit SQLite синхронізує журнал і файл БД на диск	експ. 4: аварійне завершення процесу після commit – бал 50.0, після flush без commit – 95.0

8. Міграції Alembic

Після `alembic init migrations` ревізії створено командами `alembic revision --autogenerate --rev-id 0001 -m "create groups and students"` і `... --rev-id 0002 -m "add student email"` (після додавання поля `email` до моделі). Autogenerate додав колонку й UNIQUE, але не порівнює CHECK-обмеження, тому формат email дописано вручну. На SQLite зміна таблиці виконується через batch («створити нову таблицю → скопіювати дані → замінити»):

```
def upgrade() -> None:
    with op.batch_alter_table("students", schema=None) as batch_op:
        batch_op.add_column(sa.Column("email", sa.String(length=255), nullable=True))
        batch_op.create_unique_constraint(batch_op.f("uq_students_email"), ["email"])
        batch_op.create_check_constraint(
            op.f("ck_students_email_format"), "email IS NULL OR email LIKE '%_@_%._%'"
        )

def downgrade() -> None:
    with op.batch_alter_table("students", schema=None) as batch_op:
        batch_op.drop_constraint(op.f("ck_students_email_format"), type_="check")
        batch_op.drop_constraint(batch_op.f("uq_students_email"), type_="unique")
        batch_op.drop_column("email")
```

```
PS C:\Github\prof_python\Lab_07> alembic history
0001 -> 0002 (head),
<base> -> 0001,
PS C:\Github\prof_python\Lab_07> python tools/sql.py students.db "SELECT last_name, email FROM students WHERE email IS
NOT NULL ORDER BY id"
| last_name     | email                        |
|---------------|------------------------------|
| Коваленко     | olena.kovalenko@example.com  |
| Бондар        | iryna.bondar@example.com     |
| Левандівський | nazar@example.com            |
(3 рядки)
PS C:\Github\prof_python\Lab_07> alembic downgrade -1
INFO  [alembic.runtime.migration] Context impl SQLiteImpl.
INFO  [alembic.runtime.migration] Will assume transactional DDL.
INFO  [alembic.runtime.migration] Running downgrade 0002 -> 0001
PS C:\Github\prof_python\Lab_07> python tools/sql.py students.db "SELECT name FROM pragma_table_info('students')"
```

```
| name           |
|----------------|
| id             |
| first_name     |
| last_name      |
| average_grade  |
| group_id       |
(5 рядків)
PS C:\Github\prof_python\Lab_07> alembic upgrade head
INFO  [alembic.runtime.migration] Context impl SQLiteImpl.
INFO  [alembic.runtime.migration] Will assume transactional DDL.
INFO  [alembic.runtime.migration] Running upgrade 0001 -> 0002
PS C:\Github\prof_python\Lab_07> python tools/sql.py students.db "SELECT last_name, email FROM students WHERE email IS
NOT NULL ORDER BY id"
| last_name | email |
|-----------|-------|
(0 рядків)
PS C:\Github\prof_python\Lab_07> alembic current
INFO  [alembic.runtime.migration] Context impl SQLiteImpl.
INFO  [alembic.runtime.migration] Will assume transactional DDL.
0002 (head)
```

Рисунок 4 – Історія, адреси до та після downgrade, колонки без email, повторний upgrade, поточна ревізія

Результати перевірок. (1) Міграція зберігає дані: після `upgrade` усі 1000 рядків на місці, `email` дорівнює NULL; UNIQUE дозволяє багато NULL, але відхиляє дублікат (експеримент 6). (2) Downgrade видаляє колонку разом із даними: після повторного upgrade адрес немає (рисунок 4). Збіг схеми з моделями перевіряє тест `test_upgrade_head_creates_schema_identical_to_models` (те саме, що `alembic check`). (3) Batch-ревізію не можна відобразити як SQL у режимі `--sql`: потрібне живе з'єднання для reflection. (4) У `env.py` увімкнено `transactional_ddl` з явним BEGIN: штучно зламана ревізія 0003 лишила БД на 0002 без нової колонки, а зі стандартним `env.py` номер версії теж лишився 0002, але колонка `note` залишилась у таблиці – схема розійшлась із записаною версією (експеримент 7).

9. Integration tests

Тести перевіряють взаємодію Repository/Service + SQLAlchemy + SQLite на окремій тестовій БД: кожен тест отримує власний файл `test_students.db` у `tmp_path` (схема – `create_all`), тож тести незалежні й не торкаються `students.db`. Фікстура `query` читає дані окремим DB-API з'єднанням – лише те, що справді зафіксовано в БД, а не стан об'єкта в сесії. Міграції тестуються через `alembic.command` на такій самій окремій БД.

```python
@pytest.fixture
def engine(db_path: Path) -> Iterator[Engine]:
    engine = make_engine(f"sqlite:///{db_path}")
    Base.metadata.create_all(engine)
    yield engine
    engine.dispose()


def test_database_failure_in_the_middle_rolls_back_updates_already_sent(
    school: StudentService,
    session: Session,
    group_of: Callable[[int], str],
    monkeypatch: pytest.MonkeyPatch,
) -> None:
    target = group_id(school, "PI-23")
    real_flush = session.flush
    calls: list[int] = []

    def failing_second_flush(*args: Any, **kwargs: Any) -> None:
        calls.append(1)
        if len(calls) == 2:
            raise OperationalError("UPDATE students", {}, Exception("disk I/O error"))
```

```
        real_flush(*args, **kwargs)

    monkeypatch.setattr(session, "flush", failing_second_flush)
    with pytest.raises(OperationalError):
        transfer_students(session, [1, 2, 3], target)
    assert len(calls) == 2
    assert [group_of(1), group_of(2), group_of(3)] == ["PI-21", "PI-21", "PI-22"]
```

Таблиця 4 – Файли тестів

Файл	Тестів	Що перевіряє
test_validation.py	47	правила для імені, групи, балу, email
test_models.py	13	ORM-моделі, relationship, каскад, orphan removal
test_constraints.py	15	CHECK, UNIQUE, FOREIGN KEY, PRAGMA на рівні БД
test_group_repository.py	12	GroupRepository, агрегація по групах
test_student_repository.py	27	CRUD Student, пошук, filter_by_group, сортування, пагінація
test_service.py	40	бізнес-логіка, середній бал групи, найкращий студент, видалення
test_transactions.py	13	transfer_students, commit, rollback, ізоляція
test_dbapi.py	9	параметризований пошук через sqlite3, SQL injection
test_migrations.py	12	upgrade, downgrade, збереження даних, збіг схеми з моделями
test_cli.py	24	команди CLI, коди виходу, повідомлення про помилки

```
PS C:\Github\prof_python\Lab_07> pytest -v
============================= test session starts =============================
platform win32 -- Python 3.12.10, pytest-9.2.0, pluggy-1.7.0 -- C:\Github\prof_python\Lab_07\.venv\Scripts\python.exe
rootdir: C:\Github\prof_python\Lab_07
configfile: pyproject.toml
testpaths: tests
plugins: cov-7.1.0
collecting ... collected 212 items

tests/test_cli.py::test_seed_creates_groups_and_students PASSED          [  0%]
tests/test_cli.py::test_seed_twice_does_not_duplicate_data PASSED        [  0%]
tests/test_cli.py::test_groups_report_shows_counts_and_averages PASSED   [  1%]
... ще рядків: 204 ...
tests/test_validation.py::test_invalid_email_is_rejected[a@b] PASSED     [ 98%]
tests/test_validation.py::test_invalid_email_is_rejected[@b.c] PASSED    [ 98%]
tests/test_validation.py::test_invalid_email_is_rejected[a@.c] PASSED    [ 99%]
tests/test_validation.py::test_invalid_email_is_rejected[a b@c.d] PASSED [ 99%]
tests/test_validation.py::test_invalid_email_is_rejected[5] PASSED       [100%]

============================= 212 passed in 8.43s =============================
PS C:\Github\prof_python\Lab_07> pytest -q --cov=student_manager --cov-branch --cov-report=term-missing
........................................................................ [ 33%]
........................................................................ [ 67%]
.....................................................................    [100%]
============================== tests coverage ==============================
_____________ coverage: platform win32, python 3.12.10-final-0 _____________

Name                                 Stmts   Miss Branch BrPart  Cover   Missing
--------------------------------------------------------------------------------
src\student_manager\__init__.py          1      0      0      0   100%
src\student_manager\database.py         23      0      4      1    96%   19->21
src\student_manager\dbapi.py            20      0      0      0   100%
src\student_manager\exceptions.py       16      0      0      0   100%
src\student_manager\main.py            130      0     22      0   100%
src\student_manager\models.py           40      0      0      0   100%
src\student_manager\repositories.py     79      0     10      0   100%
src\student_manager\services.py        120      0     24      0   100%
src\student_manager\validation.py       33      0     14      0   100%
--------------------------------------------------------------------------------
TOTAL                                  462      0     74      1    99%
Required test coverage of 90.0% reached. Total coverage: 99.81%
212 passed in 10.19s
PS C:\Github\prof_python\Lab_07> mypy
Success: no issues found in 25 source files
PS C:\Github\prof_python\Lab_07> ruff check .
All checks passed!
```

Рисунок 5 – Результат pytest -v (212 тестів пройдено за 8.43 с; показано початок і кінець виводу з 222 рядків, пропущено 204 рядків PASSED), покриття, mypy (strict = true у pyproject.toml) і ruff

10. Експерименти

Таблиця 5 – Результати experiments/run_experiments.py (кожен – на власній тимчасовій БД)

Експеримент	Результат
1. SQL injection	запит з f-string повернув 9 студентів із 9, параметризований – 0
2. FOREIGN KEY	без прагми студента з неіснуючою групою збережено (сиріт: 1), видалення групи не каскадне (3 сиріт); з прагмою – IntegrityError: FOREIGN KEY constraint failed, після видалення групи лишилось 6 студентів
3. Atomicity	переведення [1, 2, 3, 777, 5]: commit на кожному студенті – 3 переведено, DB-API з rollback – 0, transfer_students – 0
4. Durability	os._exit після commit – 50.0, після flush без commit – 95.0 (початковий бал 95.0)
5. N+1 запитів	300 груп, 3000 студентів: lazy loading – 301 запитів, 0.0639 с; selectinload – 2, 0.0131 с; GROUP BY – 1, 0.0013 с
6. Міграція з даними	1000 рядків: після upgrade 1000, email NULL – 1000; дублікат email відхилено
7. Збій міграції	«помилка посередині міграції»: env.py проєкту – схема без змін; стандартний env.py – колонка note залишилась при версії 0002
8. Вартість commit	300 оновлень: commit кожного – 1.317 с, одна транзакція – 0.075 с (у 17.6 раза швидше)
9. Індекс і агрегація	100 000 студентів, 200 пошуків за прізвищем: SEARCH students USING COVERING INDEX ix_students_last_name (last_name=?) – 0.007 с; SCAN students – 1.0953 с (у 156 разів повільніше). Середній бал: AVG у SQL – 0.0058 с, вибірка всіх об'єктів і середнє в Python – 0.6361 с (результат однаковий, 69.9417)

Пакетні зміни варто групувати в одну транзакцію (commit найдорожчий), але проміжні кроки однієї бізнес-операції фіксувати не можна (експеримент 3); агрегації та фільтри виконують у SQL, а зв'язані колекції завантажують разом.

Висновки

Систему обліку студентів (варіант 1) переведено на SQLite: дві пов'язані таблиці з PRIMARY KEY, FOREIGN KEY, UNIQUE і CHECK, SQLAlchemy ORM, Generic Repository, сервіс із транзакціями, пошук через DB-API та дві міграції Alembic. 212 integration тестів на окремій тестовій БД (99.81 % покриття з гілками) перевіряють CRUD, пошук, агрегацію, переведення з rollback, обмеження та міграції. Експерименти показали: FOREIGN KEY у SQLite вмикають явно, rollback захищає від часткових операцій, параметризовані запити відсікають SQL injection, а міграції SQLite потребують транзакційного DDL і перевірки згенерованого коду.